

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

```
if (x > y || p < q)
```

Intermediate code:

```
if x > y goto L1
```

```
if p < q goto L1
```

```
goto L2
```

```
L1: ...
```

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
-------------	--------

200: if x==y goto _	
---------------------	--

201: goto _	
-------------	--

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

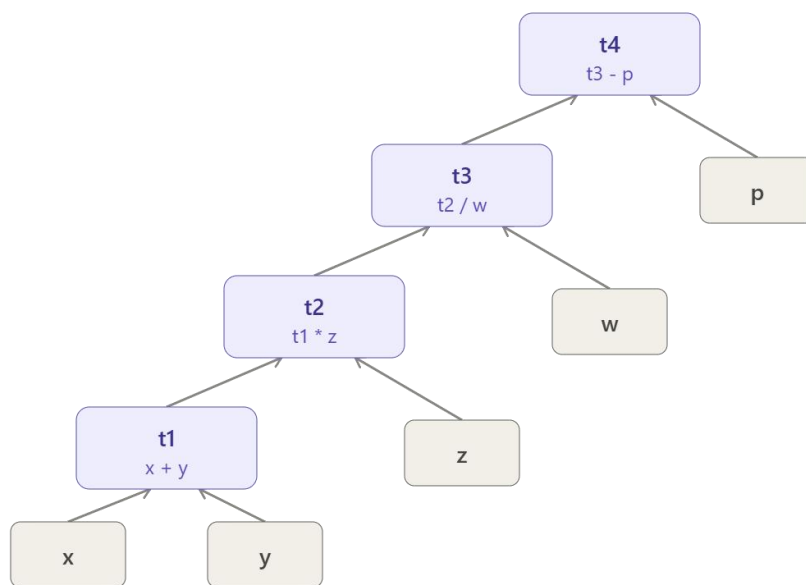
ANSWERS

Q1. Expression Tree & Role of Temporary Variables

The four three- address statements chain together into a single arithmetic expression:

$$t4 = ((x + y) * z) / w - p$$

Each t_i is not a new independent value — it's a *pointer* to a subtree result. Because t_1 feeds into t_2 , t_2 into t_3 , and t_3 into t_4 , the tree is a left-skewed (chain) tree with one leaf hanging off the right of each internal node.



Role of temporary variables:

- **Naming intermediate results.** Each t_i gives a name to a subexpression's value, so the tree/DAG structure of the expression can be expressed as a flat, linear instruction sequence — exactly what a machine (or a stack/register-based intermediate form) needs.
- **Avoiding recomputation.** Once $x + y$ is computed and stored in t_1 , every later statement that needs $(x+y)$ simply reuses t_1 instead of re-evaluating the addition. If the compiler later needs $(x+y)*z$ again elsewhere, it can reuse t_2 rather than regenerating the whole subtree.
- **Enabling optimization.** Because each temporary corresponds to exactly one node of the expression tree, the compiler can apply common-subexpression elimination, constant folding, and dead-code elimination directly on this linear code, without having to re-derive tree structure.

- **Simplifying code generation.** Register allocation, instruction scheduling, and target code generation all operate more easily on a sequence of simple binary operations than on a deeply nested expression.

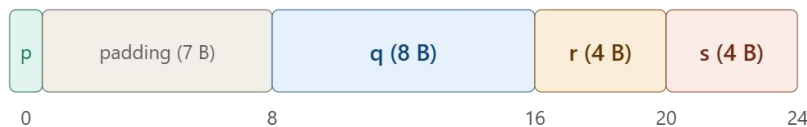
Q2. Symbol Table & Memory Allocation

Variable	Type	Size	Alignment requirement
p	char	1 B	1-byte boundary
q	double	8 B	8-byte boundary
r	int	4 B	4-byte boundary
s	float	4 B	4-byte boundary

Unaligned (packed) total: $1 + 8 + 4 + 4 = 17$ bytes.

But compilers don't pack tightly — each variable must start at an address that is a multiple of its own size (its *alignment requirement*), because most CPUs fetch multi-byte values fastest (or only correctly) when they sit on aligned boundaries.

Declaration order: p (char), q (double), r (int), s (float)



Total allocated size = 24 bytes

(17 bytes of data + 7 bytes padding, rounded up to a multiple of 8)

Interpretation:

- p (char) is placed first at offset 0 — needs only 1-byte alignment, so no padding before it.
- q (double) needs 8-byte alignment, but offset 1 isn't a multiple of 8, so the compiler inserts 7 bytes of padding and places q at offset 8.
- r (int) needs 4-byte alignment; offset 16 already satisfies that, so r sits at 16–19 with no gap.
- s (float) needs 4-byte alignment; offset 20 works fine, so s sits at 20–23.
- The overall structure's size must be a multiple of its *strictest* member alignment (8, from double), so the final size rounds up to 24 bytes.

Key takeaways:

- Alignment trades memory for speed — misaligned access can cost extra CPU cycles or even fault on some architectures.

- Declaration order matters. If variables were reordered largest-to-smallest (q, r, s, p), padding could sometimes be reduced, though here it still rounds up to 24 bytes because of the double's 8-byte requirement. As a general rule, ordering members from largest to smallest alignment minimizes total padding.
- The symbol table stores each variable's type, size, and (implicitly) its alignment so the compiler can compute exact offsets during code generation.

Q3. Optimization — Common Subexpression Elimination

```
t1 = m * n
t2 = p + q
t3 = m * n    ← identical to t1
t4 = t2 - t3
```

Step-by-step trace:

Step	Instruction	Observation
1	t1 = m * n	Computes m*n, stores in t1
2	t2 = p + q	Independent computation, unrelated to t1
3	t3 = m * n	Redundant — m and n haven't changed since step 1, so t3 always equals t1
4	t4 = t2 - t3	Uses the redundant t3

Optimization — Common Subexpression Elimination (CSE):

Since no assignment to m or n occurs between statements 1 and 3, the compiler can prove $t3 \equiv t1$ and eliminate the second multiplication entirely:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1    (t3 replaced by t1; the multiply at step 3 is deleted)
```

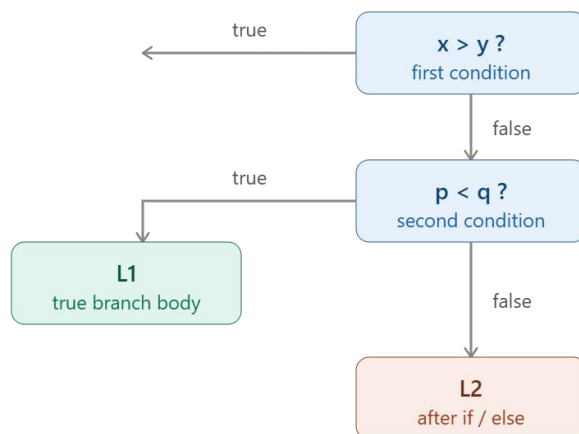
Why this helps:

- **Fewer instructions** → less code to fetch/decode/execute.
- **Fewer arithmetic operations** → especially valuable when the repeated operation is expensive (multiplication/division vs. simple addition).
- **Reduced register/memory pressure** → t3 no longer needs to be computed or stored, freeing a register.
- This is a classic *local* optimization (within a basic block), typically implemented using a **DAG (directed acyclic graph)** representation of the block, where identical subexpressions naturally collapse into a single node.

Q4. Control Flow for Short-Circuit Boolean Evaluation

For `if (x > y || p < q) ...`, the compiler generates jump-based code rather than computing a full boolean value:

```
if x > y goto L1
if p < q goto L1
goto L2
L1: ... (true branch)
L2: ... (false branch / after if)
```



Interpretation:

- The `||` operator is translated as a chain of conditional jumps, not as a boolean value computed with a bitwise/logical OR instruction.
- If `x > y` is true, control jumps directly to L1 — the second condition `p < q` is never evaluated.
- Only if `x > y` is false does execution fall through to check `p < q`.
- If both are false, control falls through to goto L2, skipping the true branch.

Why short-circuit evaluation matters:

- **Efficiency** — avoids evaluating conditions that can no longer affect the result.
- **Safety** — lets programmers write expressions like `if (ptr != NULL && ptr->value > 0)` where the second condition would crash if evaluated when the first is false; short-circuiting guarantees it's skipped.
- **Fewer instructions on average** — the common case (first condition true) exits early instead of computing every subexpression and combining them with logical operators.

Q5. Backpatching for Boolean Expression Translation

The problem backpatching solves: When generating code for a boolean expression left-to-right, the compiler often doesn't yet know the target label of a jump (e.g., where the "true" branch code will start) because that code hasn't been generated yet. Rather than inventing labels ahead of time, the compiler emits jump instructions with a blank target field, records the instruction number in a list, and fills in ("patches") the address once it becomes known.

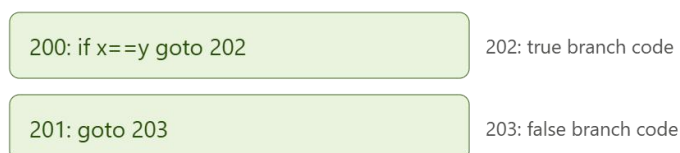
For if $x == y$ goto _ / goto _:

- 200: if $x == y$ goto _ $\rightarrow$ placed in **truelist** = {200} (this jump should go to wherever the "true" code begins)
- 201: goto _ $\rightarrow$ placed in **falselist** = {201} (this jump should go to wherever the "false" code begins)

Before backpatching



After backpatching



How it works:

1. **Emit incomplete jumps.** While generating code for a boolean expression, whenever the compiler needs a forward jump whose destination isn't known yet (because that code comes later), it emits the jump with an empty target and records the instruction's address in a list (truelist, falselist).
2. **Merge lists as the expression grows.** For compound expressions ($\&\&$, $\parallel$), the compiler merges sublists — e.g., for $B1 \parallel B2$, $B1$'s falselist gets backpatched to the start of $B2$, and the two truelists are merged into one.
3. **Patch when the target is known.** Once code generation reaches the point where the true-branch (or false-branch) code actually begins, the compiler calls a `backpatch(list, target)` procedure that goes through every instruction number in the list and fills in that target address — turning 200: if $x == y$ goto _ into 200: if $x == y$ goto 202.

Why it matters:

- It allows single-pass code generation for control structures (if, while, $\&\&$, $\parallel$) without needing a second pass to resolve labels, and without pre-inventing symbolic labels for every branch.

- It's essential for translating short-circuit boolean expressions efficiently, since the true/false destinations often depend on code that is generated *after* the condition.
- It keeps the intermediate representation compact — instructions only carry the information available at the time they're generated, with the compiler cleaning up the loose ends afterward.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand